

Karel Midpoint Program — Detailed Explanation with Code Blocks

Goal

We need Karel to **find the midpoint** of the bottom row (1st Street), place **one beeper** there, and leave the world clean.

Complete Code (with Block-by-Block Explanations)

Logic Behind how its working

For a grid of width n , the sequence of avenues to place beepers on the first row should be:

- n (Last)
- 1 (First)
- $n - 1$ (Second to Last)
- 2 (Second)
- $n - 2$ (Third to Last)
- 3 (Third)
- ... and so on, moving inwards.

Imports

```
from stanfordkarel import *
```

We import the Stanford Karel package so we can use commands like `move()`, `put_beeper()`, `turn_left()`, etc.

Main Program Structure

```
def main():
```

This is the **entry point**. When `run_karel_program()` is called, it runs `main()`.

Helper Function: Move to Target Avenue

```
def move_to_avenue(target_avenue, current_avenue):  
    while current_avenue != target_avenue:  
        if current_avenue < target_avenue:  
            while not facing_east():  
                turn_left()  
            move()  
            current_avenue += 1  
        else:  
            while not facing_west():  
                turn_left()  
            move()  
            current_avenue -= 1  
    return current_avenue
```

What this does:

- Safely moves Karel **horizontally** to a specific avenue.
- Checks direction: if the target is to the right, face east; if to the left, face west.
- Steps forward and updates current_avenue until reaching the target.

This is important because we don't want to hardcode back-and-forth moves; we let Karel "navigate" intelligently.

Step 1: Count the Width

```
width = 1  
current_avenue = 1  
while front_is_clear():  
    move()  
    width += 1  
    current_avenue += 1
```

Logic:

- We **count how many avenues** there are on 1st Street.
- Karel moves east, incrementing width and current_avenue until it hits the eastern wall.

Example:

- In a 5x5 grid, Karel moves 4 times → width = 5.

Step 2: Place Beeper at Far Right

```
put_beeper()
```

Logic:

- Once at the far right (last avenue), **drop a beeper**.

Step 3: Go to Far Left and Place Beeper

```
turn_around()
```

```
current_avenue = move_to_avenue(1, current_avenue)
```

```
put_beeper()
```

Logic:

- Turn around (face west).
- Use helper function to go back to avenue 1.
- Drop a beeper on the far left.

Step 4: Bounce Inward Alternating Right & Left

```
for i in range(1, (width + 1) // 2):
```

```
    # Go to right side: width - i
```

```
    current_avenue = move_to_avenue(width - i, current_avenue)
```

```
    put_beeper()
```

```
    # Avoid double center placement on odd widths
```

```
    if width % 2 == 0 or i < width // 2:
```

```
        # Go to left side: i + 1
```

```
        current_avenue = move_to_avenue(i + 1, current_avenue)
```

```
        put_beeper()
```

Logic:

We start filling inward:

- First from the **right**, one step in.
- Then from the **left**, one step in.
- Repeat, alternating right → left → right → left.

Important:

- For odd-width grids (like 5), we stop just before placing two beepers in the center.
- That's why we check:
- `if width % 2 == 0 or i < width // 2:`
- → Only go left again if:
- The width is even, **or**
- We haven't yet reached the middle.

Step 5: Face East to Finish

```
while not facing_east():  
    turn_left()
```

Logic:

- To neatly finish, Karel faces east when done.

Helper Function: Turn Around

```
def turn_around():  
    turn_left()  
    turn_left()
```

Logic:

- Just a simple 180° turn.

Run the Program

```
if __name__ == "__main__":
    run_karel_program()
```

This tells Python to **start the Karel simulation** when you run the file.

Example: 5x5 World

Let's walk it:

Step	Karel's Position	Action
Count width	5	Count width = 5
Place	5	Beeper at avenue 5
Go left	1	Beeper at avenue 1
Bounce R	4	Beeper at avenue 4
Bounce L	2	Beeper at avenue 2
Middle	3	Final beeper at 3

Result: Only **one beeper** remains at the center when cleanup is done.

=====END=====